

第三章 动态规划

dynamic programming

1. 动态规划一般原理(与分治对比), Bellman, OSP
2. 动态规划设计步骤: 矩阵连乘
3. 如何设计动态规划算法: 子结构和策略
最长公共子序列, 最大子段和, 最长递增子序列
4. 背包问题 (动态规划与贪心对比)
5. 资源分配问题
6. 最短路问题、旅行商问题
7. 凸多边形最优三角剖分

背景知识

- 问题求解的分类
 - 求任意解，求最优解
- 求最优解的算法大都具有指数级的复杂度，因此好的方法很重要，有一种多项式时间的复杂度算法——**动态规划**

发展及研究内容

- **动态规划**(*dynamic programming*)是运筹学的一个分支, 20世纪50年代初美国数学家**R. E. Bellman**等人在研究**多阶段决策过程**(*multistep decision process*)的**优化问题**时, 提出了著名的**最优化原理**(*principle of optimality*), **把多阶段过程转化为一系列单阶段问题, 逐个求解, 创立了解决这类过程优化问题的新方法——动态规划。**
- 动态规划问世以来, 在经济管理、生产调度、工程技术和最优控制等方面得到了**广泛的应用**。例如最短路线、资源分配、设备更新等问题, 用动态规划比用其它方法求解更为方便。

发展及研究内容

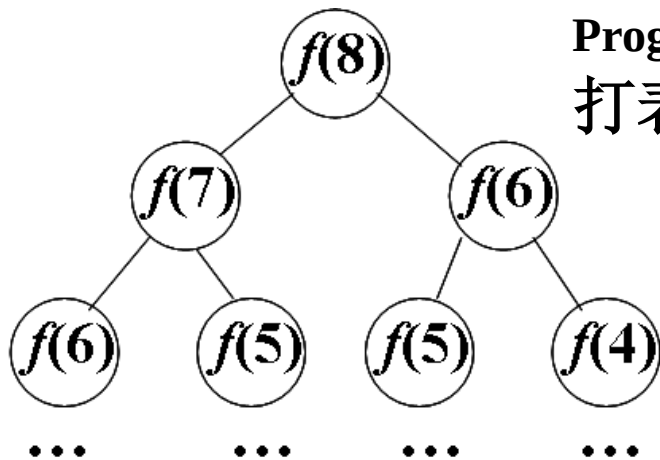
- 虽然动态规划主要用于求解以时间划分阶段的动态过程的优化问题，但是一些与时间无关的静态规划(如线性规划、非线性规划)，可以人为地引进时间因素，把它视为多阶段决策过程，也可以用动态规划方法方便地求解。

动态规划与分治

- 分治的过程：
分解—递归解子问题—合并
- 若有大量重复子问题，则不宜分治
- 举例：Fibonacci数的计算

$$f(n) = f(n-1) + f(n-2), f(1)=1, f(0)=0.$$

输入 n , 输出 $f(n)$.



Programming的含义：
打表记录中间结果

递归：

```
int f(int n)
{ if(n<2) return(1);
  return(f(n-1)+f(n-2));}
 $2^{O(n)}$ 时间,  $O(n)$ 空间
```

动态规划：

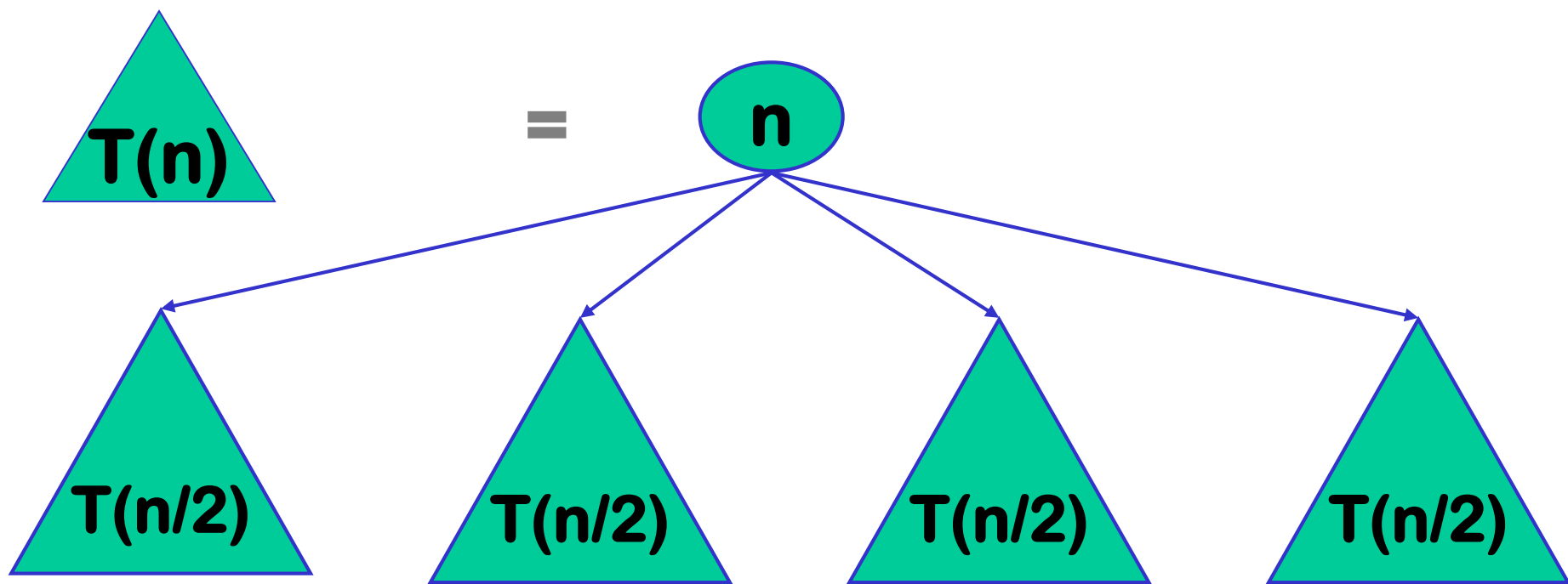
```
f[1]=1;f[0]=0;
for(i=2,i<n,i++)
    f[i]=f[i-1]+f[i-2];
 $O(n)$ 时间,  $O(n)$ 空间
f=1; b=0;i=1;
while(i++<n){
    temp=f;f=f+b;b=temp;}
 $O(n)$ 时间,  $O(1)$ 空间
```

发展及研究内容

- 虽然动态规划主要用于求解以时间划分阶段的动态过程的优化问题，但是一些与时间无关的静态规划(如线性规划、非线性规划)，可以人为地引进时间因素，把它视为多阶段决策过程，也可以用动态规划方法方便地求解。

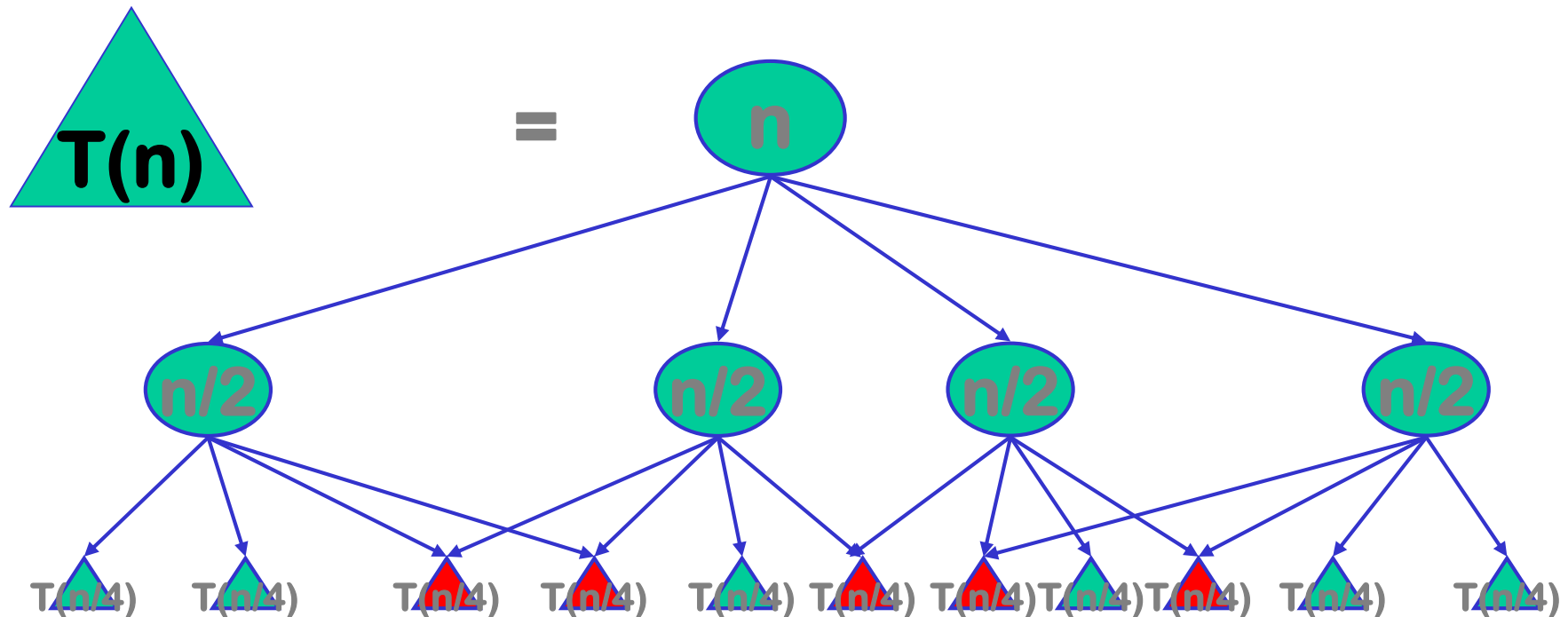
算法基本思想

- 动态规划算法与分治法类似，其基本思想是将待求解问题分解成若干个子问题



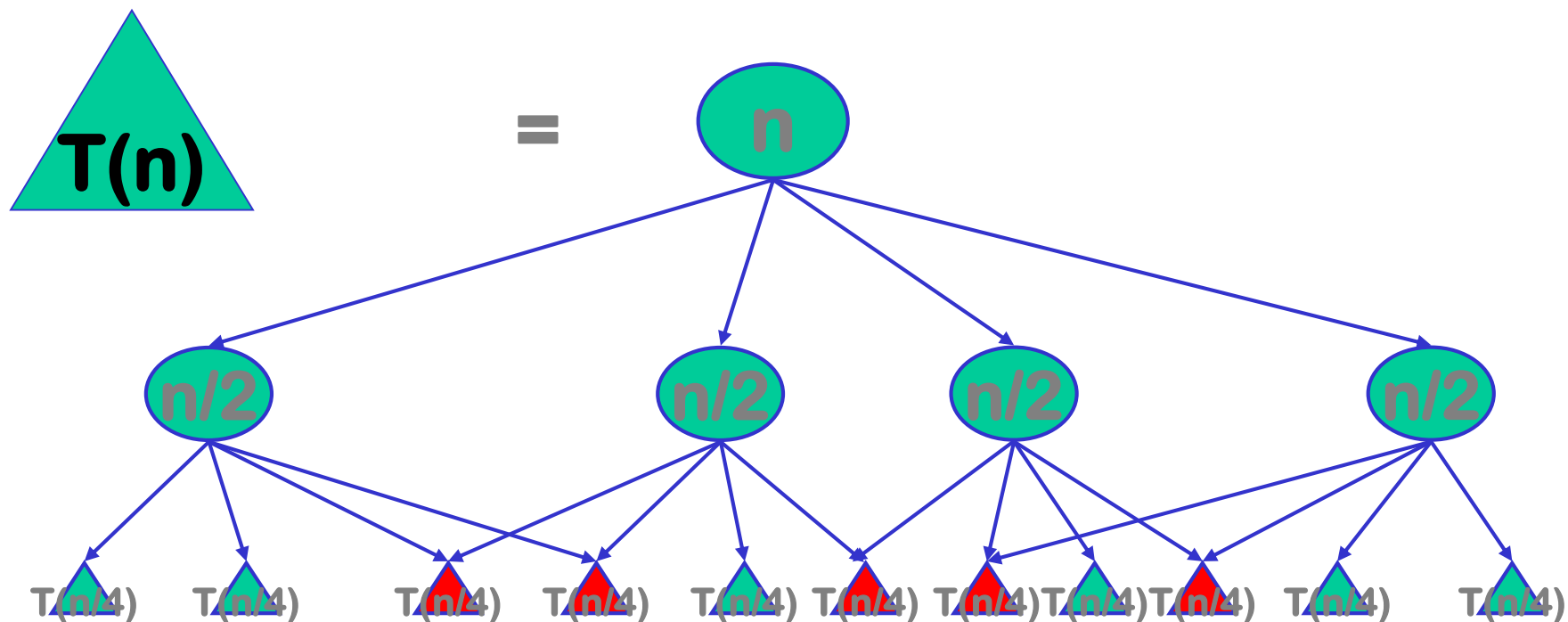
算法基本思想

- 但是经分解得到的子问题往往不是互相独立的。
不同子问题的数目常常只有多项式量级。



算法基本思想

- 如果能够保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。



与分治法的异同点

- 动态规划的思想实质是**分治思想**和**解决冗余**。
- 与分治法类似的是，将原问题分解成若干个子问题，先求解子问题，然后从这些子问题的解得到原问题的解。
- 与分治法不同的是，**经分解的子问题往往不是互相独立的**。若用分治法来解，有些共同部分（子问题或子子问题）被重复计算了很多次。

基本思想

- 如果能够保存已解决的子问题的答案，在需要时再查找，这样就可以避免重复计算、节省时间。动态规划法用一个表来记录所有已解的子问题的答案。
- 这就是动态规划法的基本思路。具体的动态规划算法多种多样，但它们具有相同的填表方式。

求解步骤

- 1、找出最优解的性质，并刻画其结构特征；
- 2、递归地定义最优值（写出动态规划方程）；
- 3、以自底向上的方式计算出最优值；
- 4、根据计算最优值时记录的信息，构造最优解。

注：

- 步骤1-3是动态规划算法的基本步骤。在只需要求出**最优值**的情形，步骤4可以省略
- 若需要求出问题的一个**最优解**，则必须执行步骤4，**步骤3中记录的信息是构造最优解的基础**

适用条件

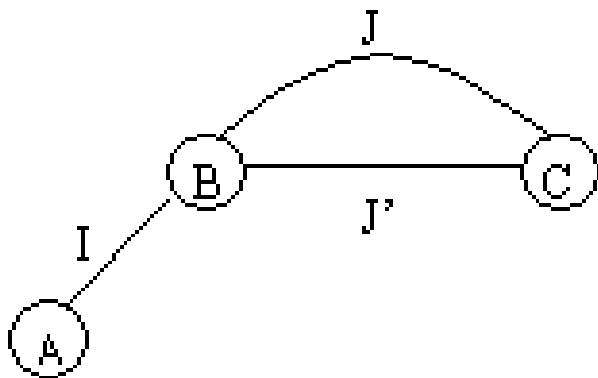
动态规划法的有效性依赖于问题本身所具有的**两个重要性质**

- **最优子结构**：当问题的最优解包含了其子问题的最优解时，称该问题具有最优子结构性质。
- **重叠子问题**：在用递归算法自顶向下解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。动态规划算法正是利用了这种子问题的重叠性质，**对每一个子问题只解一次，而后将其解保存在一个表格中，在以后尽可能多地利用这些子问题的解。**

最优性原理及举例

- Bellman给出这个原理作为动态规划的适用条件，后来Morin在1982年证明了这只是一个充分条件而非必要条件。
- 最优性原理又称为最优子结构性质：
 - 一个最优策略的子策略总是最优的。

最优性原理及举例



例如图2中，若路线I和J是A到C的最优路径，则根据最优化原理，路线J必是从B到C的最优路线。这可用反证法证明：假设有另一路径J'是B到C的最优路径，则A到C的路线取I和J'比I和J更优，矛盾。从而证明J必是B到C的最优路径。

最优性原理及举例

- 最优化原理是动态规划的基础。
- 任何问题，如果失去了最优化原理的支持，就不可能用动态规划方法计算。

设计技巧

- 动态规划的设计技巧：阶段的划分和状态的表示；
- 在动态规划的设计过程中，**阶段的划分**和**状态的表示**是非常重要的两步，这两步会直接影响该问题的计算复杂性，有时候阶段划分或状态表示的不合理还会使得动态规划法不适用。

存在的问题

- 空间溢出的问题，是动态规划解决问题时一个普遍遇到的问题；
- 动态规划需要很大的空间以存储中间产生的结果，这样可以使包含同一个子问题的所有问题共用一个子问题解，从而体现动态规划的优越性，但这是以牺牲空间为代价的，为了有效地访问已有结果，数据也不易压缩存储，因而空间矛盾是比较突出的。

第三章 动态规划

dynamic programming

1. 动态规划一般原理(与分治对比), Bellman, OSP
2. 动态规划设计步骤: 矩阵连乘
3. 如何设计动态规划算法: 子结构和策略
最长公共子序列, 最大子段和, 最长递增子序列
4. 背包问题 (动态规划与贪心对比)
5. 资源分配问题
6. 最短路问题、旅行商问题
7. 凸多边形最优三角剖分

应用1：矩阵连乘问题

矩阵连乘问题([王])

- 输入: 给定矩阵 $A_1, A_2, \dots, A_n$, A_i 与 A_{i+1} 可乘
- 输出: 计算量最小的乘法次序
- 输入样例: $A_1(10 \times 100 \text{阶}), A_2(100 \times 5 \text{阶}), A_3(5 \times 50 \text{阶})$,
- 两种计算次序: $((A_1 \times A_2) \times A_3), (A_1 \times (A_2 \times A_3))$
 - $A_1 \times A_2$ 的计算量: **$10 \times 100 \times 5$** (乘法次数)
 - $((A_1 \times A_2) \times A_3) : 10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$
 - $(A_1 \times (A_2 \times A_3)) : \mathbf{100 \times 5 \times 50} + \mathbf{10 \times 100 \times 50} = 75000$
- 样例输出: **$((A_1 \times A_2) \times A_3)$**

回顾矩阵相乘

[illegible]

- 单个乘法次数:
- 单个加法次数:
- 总的乘法次数:
- 总的加法次数:

■ **n**

■ **n-1**

■ **$m * n * l$**

- $m^*(n-1)^*l$

矩阵连乘积问题

给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中 A_i 与 A_{i+1} 是可乘的， $i=1, 2, \dots, n-1$ 。

如何确定计算矩阵连乘积的计算次序，使得依此次序计算矩阵连乘积需要的数乘次数最少。

矩阵连乘积问题

- 给定n个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中 A_i 与 A_{i+1} 是可乘的， $i = 1, 2, \dots, n-1$ 。考察这n个矩阵的连乘积

$$A_1 A_2 \dots A_n$$

- 由于矩阵乘法满足结合律，所以计算矩阵的连乘可以有許多不同的计算次序。这种计算次序可以用加括号的方式来确定。
- 若一个矩阵连乘积的计算次序完全确定，也就是说该连乘积已完全加括号，则可以依此次序反复调用2个矩阵相乘的标准算法计算出矩阵连乘积

矩阵连乘问题

给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中 A_i 与 A_{i+1} 是可乘的， $i=1, 2, \dots, n-1$ 。如何确定计算矩阵连乘积的计算次序，使得依此次序计算矩阵连乘积需要的数乘次数最少。

◆**穷举法**：列举出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。

算法复杂度分析：

对于 n 个矩阵的连乘积，设其不同的计算次序为 $P(n)$ 。

由于每种加括号方式都可以分解为两个子矩阵的加括号问题： $(A_1 \dots A_k)(A_{k+1} \dots A_n)$ 可以得到关于 $P(n)$ 的递推式如下：

$$P(n) = \begin{cases} 1 & n=1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n>1 \end{cases} \Rightarrow P(n) = \Omega(4^n / n^{3/2})$$

问题

- 如何分解问题？
- 子问题的递归结构是什么？
- 矩阵链乘问题满足最优性原理？

问题

- 矩阵链乘问题是否满足最优性原理？
- 设 $A[i:j]$ 为 $A_i A_{i+1} \dots A_j$ 链乘的一个最优括号方案，设 $A[i:j]$ 的最优次序中含有二个子链 $A[i:k]$ 和 $A[k+1:j]$ ，则 $A[i:k]$ 和 $A[k+1:j]$ 是否是最优次序？

矩阵连乘问题

矩阵链乘问题满足最优性原理

记 $A[i:j]$ 为 $A_i A_{i+1} \dots A_j$ 链乘的一个最优括号方案

设 $A[i:j]$ 的最优次序中含有二个子链 $A[i:k]$ 和 $A[k+1:j]$ ，则 $A[i:k]$ 和 $A[k+1:j]$ 也是最优的。

分析最优解结构、建立递推关系

假设定好了 $A_1 \dots A_n$ 一个乘法次序 P

- 用 $A[i:j]$ 记连乘积 $A_i \dots A_j$, 相应计算量 $T[i,j]$
- 设 P 最后乘法在 A_k 后断开, 即 $A[1:k] \times A[k+1:n]$
那么 P 的计算量为 $T[1,k] + T[k+1,n] + q_0 \times q_k \times q_n$.
- 若 P 最优, 则 P 在 $A[1:k]$ 和 $A[k+1:n]$ 上也最优

最优子结构性质 : 最优策略的子策略也是最优.

设 $A[i:j]$ 的最小计算量为 $m[i,j]$, 那么

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1} q_k q_j\} & i < j \end{cases}$$

最优值与最优解的区别

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

- 输入: 给定矩阵 $A_1, A_2, \dots, A_n$ 的维数序列:
 $q_0, q_1, \dots, q_n$, 即 A_i 是 $q_{i-1} \times q_i$ 阶矩阵
- 输出: 计算量最小的乘法次序
- **最优解**是要输出的次序
最优值是最优解的计算量

DP适用条件和设计步骤

- OSP: 最优策略的子策略也是最优.
- 重叠子问题性质: 记录中间结果.

设计步骤

- 1) 描述最优解的结构
- 2) 递归定义最优解
- 3) 自底向上计算最优值
- 4) 由计算结果构造最优解

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

观察最优值计算

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

- 需要计算的是 $m[1, n]$, 但没法直接计算

- $m[1, 1] = m[2, 2] = \dots = m[n, n] = 0$

- $m[1, 2] = ?$

$$m[1, 2] = q_0 \times q_1 \times q_2, m[2, 3] = q_1 \times q_2 \times q_3, m[3, 4], \dots$$

- $m[1, 3] = ?$

$$\min\{ m[1, 1] + m[2, 3] + q_0 \times q_1 \times q_3, m[1, 2] + m[3, 3] + q_0 \times q_2 \times q_3 \}$$

- 自底向上计算; 表格化方法

计算最优值图示

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

设维数序列为(30, 35, 15, 5, 10, 20, 25)

m		i					
		1	2	3	4	5	6
j	1						
	2						
	3						
	4						
	5						
	6						

m		i					
		1	2	3	4	5	6
j	1						
	2						
	3						
	4						
	5						
	6						

计算最优值图示

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

(30, 35, 15, 5, 10, 20, 25)

$$m[1, 2] = q_0 \times q_1 \times q_2 = 30 \times 35 \times 15 = 15750$$

m		i					
		1	2	3	4	5	6
j	1						
	2						
	3						
	4						
	5						
	6						

m		i					
		1	2	3	4	5	6
j	1	0	15750				
	2		0	2625			
	3			0	750		
	4				0	1000	
	5					0	5000
	6						0

计算最优值图示

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

$$m[1, 3] = \min \{ m[1, 1] + m[2, 3] + q_0 \times q_1 \times q_3, m[1, 2] + m[3, 3] + q_0 \times q_2 \times q_3 \}$$

$$= \min \{ 0 + 2625 + 30 \times 35 \times 5, 15750 + 0 + 30 \times 15 \times 5 \} = \min \{ 18000, 7875 \}$$

m		i					
		1	2	3	4	5	6
j	1						
	2						
	3						
	4						
	5						
	6						

m		(30, 35, 15, 5, 10, 20, 25)					
		1	2	3	4	5	6
j	1	0	15750	7875			
	2		0	2625	4375		
	3			0	750	2500	
	4				0	1000	3500
	5					0	5000
	6						0

计算最优值图示

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

$$\begin{aligned} m[1,4] &= \min \{m[1,1]+m[2,4]+q_0 \times q_1 \times q_4, \\ &\quad m[1,2]+m[3,4]+q_0 \times q_2 \times q_4, \\ &\quad m[1,3]+m[4,4]+q_0 \times q_3 \times q_4\} \\ &= \min \{0 + 4375 + 30 \times 35 \times 10, \\ &\quad 15750 + 750 + 30 \times 15 \times 10, \\ &\quad 7875 + 0 + 30 \times 5 \times 10\} \\ &= \min \{9375, 21000, 14875\} \end{aligned}$$

m		(30, 35, 15, 5, 10, 20, 25)					
		1	2	3	4	5	6
j	1	0	15750	7875	9375		
	2		0	2625	4375	7125	
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

计算最优值图示

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

$$\begin{aligned} m[1, 5] &= \min \{ 9375 + 0 + 30 \times 10 \times 20, 7875 + 1000 + 30 \times 5 \times 20, \\ &\quad 15750 + 2500 + 30 \times 15 \times 20, 0 + 7125 + 30 \times 35 \times 20 \} \\ &= \min \{ 15750, 11875, 27250, 28125 \} \end{aligned}$$

m		i					
		1	2	3	4	5	6
j	1						
	2						
	3						
	4						
	5						
	6						

m		(30, 35, 15, 5, 10, 20, 25)					
		1	2	3	4	5	6
j	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

计算最优值算法

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

输入: n 和维数序列 $(q_0, q_1, \dots, q_n)$

1. 对 $i = 1$ 到 n , $m[i, i] = 0$,
2. 对 $r = 1$ 到 $n-1$
3. 对 $i = 1$ 到 $n-r$
4. $j = i + r$; $m[i, j] = \text{INF}$;
5. 对 $k = i$ 到 $j-1$
6. $t = m[i, k] + m[k+1, j] + q_{i-1} \times q_k \times q_j$,
7. 若 $m[i, j] > t$, 则 $m[i, j] = t$
8. 输出: $m[1, n]$

INF 如何处理? 如何构造最优解?

计算最优值同时标记分断点

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

输入: n 和维数序列 $(q_0, q_1, \dots, q_n)$

1. 对 $i = 1$ 到 n , $m[i, i] = 0$,
2. 对 $r = 1$ 到 $n-1$
3. 对 $i = 1$ 到 $n-r$
4. $j = i + r$; $s[i, j] = i$;
5. $m[i, j] = m[i, i] + m[i+1, j] + q_{i-1} \times q_i \times q_j$;
6. 对 $k = i + 1$ 到 $j-1$
7. $t = m[i, k] + m[k+1, j] + q_{i-1} \times q_k \times q_j$;
8. 若 $m[i, j] > t$, 则 $m[i, j] = t$; $s[i, j] = k$;

输出: s // $s[i, j]$ 是计算 $[i: j]$ 段的分断点

构造最优解

Traceback(1, n, s) //输出最优解, $s[i,j]$ 是 $[i:j]$ 的最优分断点

Traceback(i, j, s) //[C]

1. 若 $i == j$, 打印 “A”, i
2. 否则 打印 “(”
3. Traceback(i, s[i,j], s)
4. Traceback(s[i,j]+1, j, s)
5. 打印 “)”

10, 100, 5, 50

$s[1,3]=2$

输出样例:

((A1 A2) A3)

Traceback2(i,j,s) //[王], 书中解释不匹配

1. 若 $i == j$ 返回
2. Traceback2(i, s[i,j], s)
3. Traceback2(s[i,j]+1, j, s)
4. 打印 “A”, i , “,”, $s[i,j]$, “× A”, $s[i,j]+1$, “,”, j

输出样例

A 1,1 × A 2,2

A 1,2 × A 3,3

DP适用条件和设计步骤

- **OSP**: 最优策略的子策略也是最优.
- 重叠子问题性质: 记录中间结果.

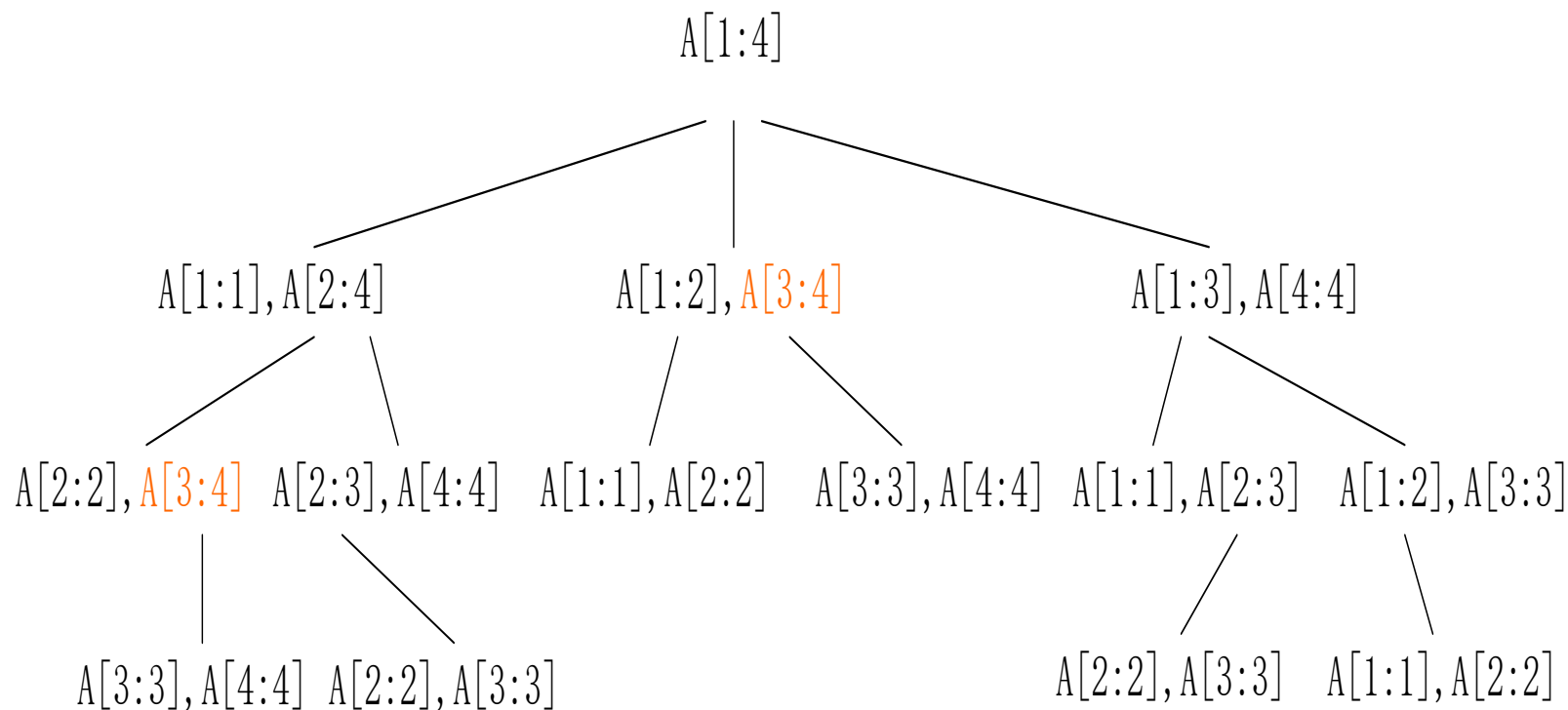
设计步骤

- 1) 描述最优解的结构
- 2) 递归定义最优解
- 3) 自底向上计算最优值 → 自顶向下
- 4) 由计算结果构造最优解

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + q_{i-1}q_kq_j\} & i < j \end{cases}$$

重叠子问题

自顶向下计算 $m[1][4]$ 过程如下：



设置备忘录

备忘录方法: 自顶向下***

- MEMOIZED-MATRIX-CHAIN

1. 对所有 i, j , $m[i, j] = 0$
2. $LU(1, n)$ //LookUp

- $LU(i, j)$

1. 若 $m[i, j] > 0$, 返回 $m[i, j]$
2. 若 $i = j$, 返回 0
3. $s[i, j] = i$; $m[i, j] = LU(i, i) + LU(i+1, j) + q_{i-1} \times q_i \times q_j$;
4. 对 $k = i + 1$ 到 $j-1$
5. $t = LU(i, k) + LU(k+1, j) + q_{i-1} \times q_k \times q_j$,
6. 若 $m[i, j] > t$, 则 $m[i, j] = t$; $s[i, j] = k$;
7. 返回 $m[i][j]$

自底向上与自顶向下的比较

- 动态规划方法采用自底向上
- 备忘录方法采用自顶向下
- 都能解决重叠子问题
- 当所有子问题都至少要求解一次时，
用动态规划方法比较好
- 当部分子问题不用求解时，
用备忘录方法比较好
- 矩阵连乘问题宜用动态规划

练习

- 求下述四个矩阵的矩阵连乘问题。

$$A_1 = (a_{i,j})_{35 \times 40}, A_2 = (a_{i,j})_{40 \times 20},$$
$$A_3 = (a_{i,j})_{20 \times 10}, A_4 = (a_{i,j})_{10 \times 15}$$

- $m_{12} = 35 \times 40 \times 20 = 28000$
- $m_{23} = 40 \times 20 \times 10 = 8000$
- $m_{34} = 20 \times 10 \times 15 = 3000$
- $m_{13} = \min\{m_{12} + 35 \times 20 \times 10, m_{23} + 35 \times 40 \times 10\}$
- $\quad = \min\{28000 + 7000, 8000 + 14000\}$
- $\quad = 22000$
- 同样有: $m_{24} = 14000$
- $m_{14} = \min\{m_{24} + 35 \times 40 \times 15, m_{12} + m_{34} + 35 \times 20 \times 15, m_{13} + 35 \times 10 \times 15\}$
- $\quad = \min\{14000 + 21000, 28000 + 3000 + 10500, 22000 + 5250\}$
- $\quad = \min\{35000, 41500, 27250\} = 27250$
- 最佳乘法顺序为: $((A_1(A_2A_3))A_4)$

练习***

- 在一个铁路沿线顺序存放着 n 堆装满货物的集装箱。货物储运公司要将集装箱有次序地集中成一堆。规定每次只能选相邻的2堆集装箱合并成新的一堆，所需的运输费用与新的一堆中集装箱数成正比。给定各堆的集装箱数，试制定一个运输方案，使总运输费用最少。
- 5, 3, 4, 2, 3, 4, 3, 4 为了简化，只算4, 2, 3, 4

练习

4, 2, 3, 4

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i < k \leq j} \{m[i, k-1] + m[k, j] + \sum_{t=i}^j a[t]\} & i < j \end{cases}$$

	$j = 1$	2	3	4
$i = 1$				
2				
3				
4				

练习***

• 在一个圆形操场的四周顺序存放着 n 堆装满货物的集装箱。货物储运公司要将集装箱有次序地集中成一堆。规定每次只能选相邻的2堆集装箱合并成新的一堆，所需的运输费用与新的一堆中集装箱数成正比。给定各堆的集装箱数，试制定一个运输方案，使总运输费用最少。5, 3, 4, 2, 3, 4, 3, 4 为了简化，只算4, 2, 3, 4

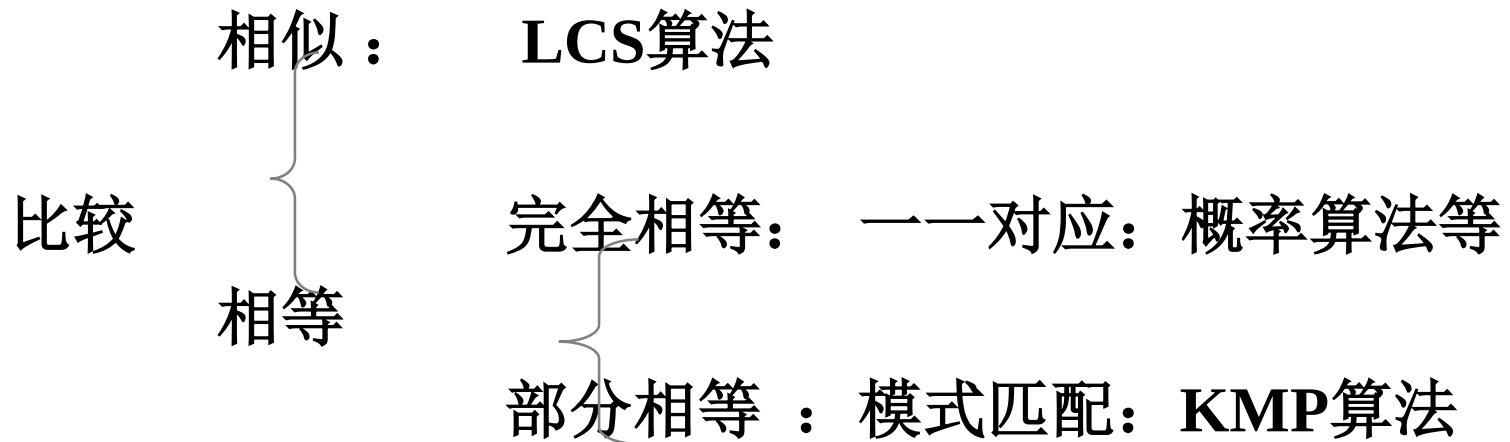
第三章 动态规划

dynamic programming

1. 动态规划一般原理(与分治对比), Bellman, OSP
2. 动态规划设计步骤: 矩阵连乘
3. 如何设计动态规划算法: 子结构和策略
最长公共子序列, 最大子段和, 最长递增子序列
4. 背包问题 (动态规划与贪心对比)
5. 资源分配问题
6. 最短路问题、旅行商问题
7. 凸多边形最优三角剖分

应用2：最长子序列问题

两个一维事物比较



最长公共子序列:

- 应用：打字比赛（规则，中外区别）
- SARS病毒
- 文件比较等等
- 两个一维事物的比较
- 定义：与子串的区别
- 算法：穷举法，动态规划法

最长公共子序列的定义

- 若给定序列 $X=\{x_1, x_2, \dots, x_m\}$ ，则另一序列 $Z=\{z_1, z_2, \dots, z_k\}$ ，是 X 的子序列是指存在一个严格递增下标序列 $\{i_1, i_2, \dots, i_k\}$ 使得对于所有 $j=1, 2, \dots, k$ 有： $z_j=x_{i_j}$ 。
- 给定2个序列 X 和 Y ，当另一序列 Z 既是 X 的子序列又是 Y 的子序列时，称 Z 是序列 X 和 Y 的公共子序列。
- 给定2个序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ ，找出 X 和 Y 的最长公共子序列。

例子

- 例如，序列 $Z=\{B, C, D, B\}$ 是序列 $X=\{A, B, C, B, D, A, B\}$ 的子序列，相应的递增下标序列为 $\{2, 3, 5, 7\}$ 。而子串则要求左右两元素在母串中为相邻。

穷举法

- 若要求两个序列X，Y的最长公共子序列，
- 先取得X，Y的所有子序列，并进行一一
- 比较，共有如下不同的组合：

$$C_m^1 + C_m^2 + \dots + C_m^m = 2^m$$

$$C_n^1 + C_n^2 + \dots + C_n^n = 2^n$$

- 共要进行不同的比较： 2^{m+n}

例子

- 代表委员建议将孔子诞辰日设为全民读书节
- 孔子诞辰日2017年3月被代表委员建议为中国的全民读书节

判断题

设序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为 $Z=\{z_1, z_2, \dots, z_k\}$ ，则

(1)若 $x_m=y_n$ ，则 $z_k=x_m=y_n$ ，
且 $z_1, z_2, \dots, z_{k-1}$ 是否为 $x_1, x_2, \dots, x_{m-1}$ 和 $y_1, y_2, \dots, y_{n-1}$ 的最长公共子序列？

(2)若 $x_m \neq y_n$ 且 $z_k \neq x_m$ ，
则 Z 是 $x_1, x_2, \dots, x_{m-1}$ 和 Y 的最长公共子序列？

(3)若 $x_m \neq y_n$ 且 $z_k \neq y_n$ ，
则 Z 是 X 和 $y_1, y_2, \dots, y_{n-1}$ 的最长公共子序列？

最优子结构性性质

- 问题：求两个序列的最长公共子序列问题是否具有最优子结构性性质？
- 当问题的最优解包含了其子问题的最优解时，称该问题具有最优子结构性性质

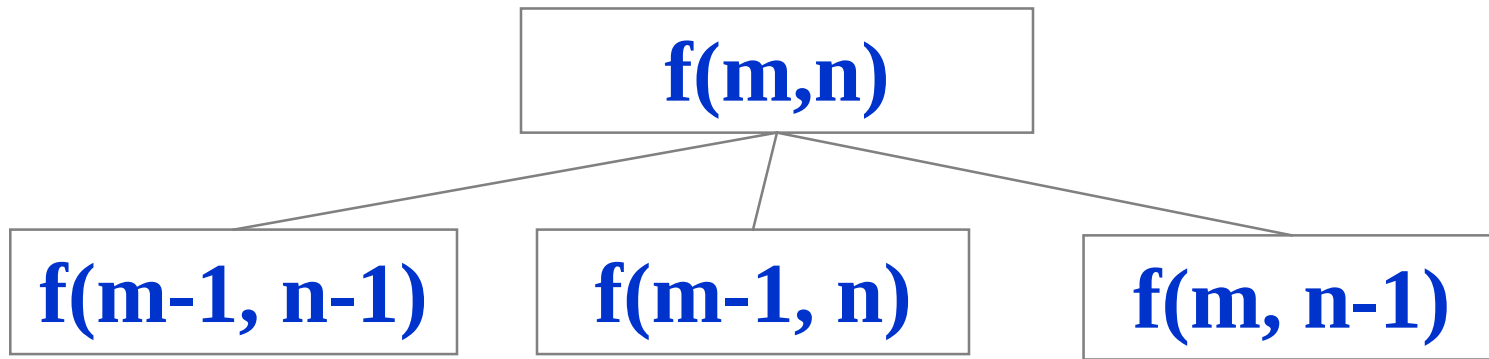
最长公共子序列的结构

设序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为 $Z=\{z_1, z_2, \dots, z_k\}$ ，则

- (1) 若 $x_m = y_n$ ，则 $z_k = x_m = y_n$ ，
且 $z_1, z_2, \dots, z_{k-1}$ 为 $x_1, x_2, \dots, x_{m-1}$ 和 $y_1, y_2, \dots, y_{n-1}$ 的最长公共子序列。
- (2) 若 $x_m \neq y_n$ 且 $z_k \neq x_m$ ，
则 Z 是 $x_1, x_2, \dots, x_{m-1}$ 和 Y 的最长公共子序列
- (3) 若 $x_m \neq y_n$ 且 $z_k \neq y_n$ ，
则 Z 是 X 和 $y_1, y_2, \dots, y_{n-1}$ 的最长公共子序列

由此可见，2个序列的最长公共子序列包含了这2个序列的前缀的最长公共子序列。因此，最长公共子序列问题具有最优子结构性质。

分析



$f(m, n)$ 表示求序列 $X=\{x_1, x_2, \dots, x_m\}$ 和
 $Y=\{y_1, y_2, \dots, y_n\}$ 的最长公共子序列

子问题的递归结构

由最长公共子序列问题的最优子结构性质建立子问题最优值的递归关系。用 $c[i][j]$ 记录序列 X 和 Y 的最长公共子序列的长度。其中, $X_i = \{x_1, x_2, \dots, x_i\}$; $Y_j = \{y_1, y_2, \dots, y_j\}$ 。当 $i=0$ 或 $j=0$ 时, 空序列是 X_i 和 Y_j 的最长公共子序列。故此时 $C[i][j]=0$ 。其它情况下, 由最优子结构性质可建立递归关系如下:

$$c[i][j] = \begin{cases} 0 & i = 0, j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

计算最长公共子序列的长度

```
void LCSLength(char x[], char y[],int m, int n)
{ /* 计算最长公共子序列的长度 */
    int L[m][n],i, j;
    for (i = 0; i <= m; i++) L[i][0] = 0;
    for (i = 0; i <= n; i++) L[0][i] = 0;
    for (i = 1; i <= m; i++)
        for (j = 1; j <= n; j++) {
            if (x[i]==y[j])
                L[i][j]=L[i-1][j-1]+1;
            else if (L[i-1][j]>= L[i][j-1])
                L[i][j]= L[i-1][j];
            else L[i][j]= L[i][j-1];
        }
    return L[m][n];
}
```

例子：填充表格

$j \backslash i$	0	1	2	3	4	5	6	
0								
1								d
2								b
3								c
4								b
5								a
6								d
7								b
		b	a	c	d	b	d	

从表中找出最长公共子序列的方法

- (1) 从 (m,n) 到 $(0,0)$
- (2) 若当前格与左边一格相同，则画 “ $\leftarrow$ ” ；
- 若当前格与上边一格相同，则画 “ $\uparrow$ ” ；
- 上两者都不符合，从当前格到左上格画 “ $\nwarrow$ ”
- (3) 从当前格向箭头方向前进一格，对此格进行(2)
- (4) 从 (m,n) 到 $(0,0)$ 的不同路径中，“ $\nwarrow$ ” 相对应的格的元素构成最长公共子序列。

找出最长公共子序 (bcbd, bcdb, badb)

i \ j	0	1	2	3	4	5	6	
0	0	0	0	0	0	0	0	
1	0	0	0	0	1	1	1	d
2	0	1	1	1	1	2	2	b
3	0	1	1	2	2	2	2	c
4	0	1	1	2	2	3	3	b
5	0	1	2	2	2	3	3	a
6	0	1	2	2	3	3	4	d
7	0	1	2	2	3	4	4	b
		b	a	c	d	b	d	

找出最长公共子序 (bcbd, bcdb, badb)

i \ j	0	1	2	3	4	5	6	
0	0	0	0	0	0	0	0	
1	0	0	0	0	1	1	1	d
2	0	1	1	1	1	2	2	b
3	0	1	1	2	2	2	2	c
4	0	1	1	2	2	3	3	b
5	0	1	2	2	2	3	3	a
6	0	1	2	2	3	3	4	d
7	0	1	2	2	3	4	4	b
		b	a	c	d	b	d	

找出最长公共子序列

i \ j	0	1	2	3	4	5	6	
0	0	0	0	0	0	0	0	
1	0	0	0	0	1	1	1	d
2	0	1	1	1	1	2	2	b
3	0	1	1	2	2	2	2	c
4	0	1	1	2	2	3	3	b
5	0	1	2	2	2	3	3	a
6	0	1	2	2	3	3	4	d
7	0	1	2	2	3	4	4	b
		b	a	c	d	b	d	

找出最长公共子序列

i \ j	0	1	2	3	4	5	6	
0	0	0	0	0	0	0	0	
1	0	0	0	0	1	1	1	d
2	0	1	1	1	1	2	2	b
3	0	1	1	2	2	2	2	c
4	0	1	1	2	2	3	3	b
5	0	1	2	2	2	3	3	a
6	0	1	2	2	3	3	4	d
7	0	1	2	2	3	4	4	b
		b	a	c	d	b	d	

Diagram illustrating the Longest Common Subsequence (LCS) problem. The table shows the DP table with indices i (rows) and j (columns). The sequence 'b' is highlighted in green, and the sequence 'a' is highlighted in blue. Red arrows indicate the path of the longest common subsequence, starting from the bottom-right cell (7,7) and moving towards the top-left cell (0,0). The path is: (7,7) → (6,7) → (6,6) → (5,6) → (5,5) → (4,5) → (4,4) → (3,4) → (3,3) → (2,3) → (2,2) → (1,2) → (1,1) → (0,1) → (0,0).

最长公共子序列

由于在所考虑的子问题空间中，总共有

$$\Theta(mn)$$

个不同的子问题，因此，用动态规划算法自底向上地计算最优值能提高算法的效率。

问题

- 如果只需要计算最长公共子序列的长度，
 - (1) 请问是否降低算法的空间复杂度？
 - (2) 若是， 请问如何降低算法的空间复杂度？

算法的改进

- 如果只需要计算最长公共子序列的长度，则算法的空间需求可大大减少。事实上，在计算 $c[i][j]$ 时，只用到数组 c 的第 i 行和第 $i-1$ 行。因此，用2行的数组空间就可以计算出最长公共子序列的长度。进一步的分析还可将空间需求减至 $O(\min(m,n))$ 。

子串比较

- **Google.com**等搜索工具的目的是从信息海洋中进行串匹配查找。
- 最著名的子串匹配算法是**KMP**算法。
 - 由D.E.Knuth与V.R.Pratt和J.H.Morris同时发现，因此人们称它为克努特——莫里斯——普拉特操作（简称KMP算法）
- **KMP**算法利用关键词内部的相似特点，节省了时间复杂度： $O(m+n)$

第三章 动态规划

dynamic programming

1. 动态规划一般原理(与分治对比), Bellman, OSP
2. 动态规划设计步骤: 矩阵连乘
3. 如何设计动态规划算法: 子结构和策略
最长公共子序列, 最大子段和, 最长递增子序列
4. 背包问题 (动态规划与贪心对比)
5. 最短路问题、旅行商问题
6. 多边形游戏问题

应用3：最大子段和

最大子段和

- 给定整数序列 $a_1, a_2, \dots, a_n$ ，求形如 $\sum_{k=i}^j a_k$ 的子段和的最大值。规定子段和为负整数时，定义其最大子段和为0，即

$$\max \left\{ 0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k \right\}$$

- 例如， $(a_1, a_2, a_3, a_4, a_5, a_6) = (-2, 11, -4, 13, -5, -2)$
最大子段和为

$$\sum_{k=2}^4 a_k = 20$$

问题

- 请问可否用分治算法求解？如果可以，如何求解？
- 请问可否用动态规划算法求解？如果可以，如何求解？

分治算法

从最大子段和问题的解的结构可以看出，它适合用分治法求解。

$a[1:n]$ 的最大子段和有三种情形：

- (1) $a[1:n]$ 的最大子段和与 $a[1:n/2]$ 的最大子段和相同。
- (2) $a[1:n]$ 的最大子段和与 $a[n/2+1:n]$ 的最大子段和相同。
- (3) $a[1:n]$ 的最大子段和为 $\sum_{k=i}^j a_k$,

且 $1 \leq i \leq n/2, n/2+1 \leq j \leq n$ 。

其中(1)和(2)这两种情形可递归求得。

对于情形(3), $a[n/2]$ 与 $a[n/2+1]$ 在最优子序列中, $a[1:n]$ 的最大子段和是 $a[1:n/2]$ 的最大子段和与 $a[n/2+1:n]$ 的最大子段和的相加

分治算法

```
int MaxSubSum (int *a, int left, int right)
{ int sum =0;
  if (left==right) sum= a[left] >0?a[left]:0;
  else { int center = (left+right)/2;
        int leftsum = MaxSubSum(a, left, center);
        int rightsum = MaxSubSum(a, center+1, right );
        int s1 =0;  int lefts =0;
        for (int i=center; i>=left; i--)
            { lefts += a[i];  if (lefts >s1) s1= lefts; }
        int s2 =0;  int rights =0;
        for (int i=center+1; i<=right; i++)
            { rights += a[i]; if (rights >s2) s2= rights; }
        sum = s1+s2;
        if (sum < leftsum) sum = leftsum;
        if (sum < rightsum) sum = rightsum;
    }
  return sum; }
```

分治算法

时间复杂度:

$$T(n) = \begin{cases} O(1) & \text{当 } n \leq 1 \\ 2T(n/2) + O(n) & \text{当 } n \geq 2 \end{cases}$$

得 $T(n) = O(n \log n)$

动态规划算法：最大子段和

请问：原问题等于什么？

如何构建动态规划方程？

- 基本思想

一子问题定义

考虑所有下标以j结束的最大子段和**b[j]**，即

$$b[j] = \max \left\{ 0, \max_{1 \leq i \leq j} \sum_{k=i}^j a_k \right\} \quad j = 1, 2, \dots, n$$

动态规划算法：最大子段和

- 基本思想

—子问题定义：考虑所有下标以j结束的 最大子段和 $b[j]$ ，即

$$b[j] = \max \left\{ 0, \max_{1 \leq i \leq j} \sum_{k=i}^j a_k \right\} \quad j = 1, 2, \dots, n$$

—原问题与子问题的关系

—子问题解的递归关系（ $b[j-1] > 0$ 时， $b[j] = b[j-1] + a_j$ ，否则 $b[j] = a_j$ ）

$$\max \left\{ 0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k \right\} = \max \left\{ 0, \max_{1 \leq j \leq n} \left\{ \max_{1 \leq i \leq j} \sum_{k=i}^j a_k \right\} \right\} = \max_{1 \leq j \leq n} \{ b[j] \}$$

$$b[j] = \max \{ b[j-1] + a_j, a_j \} \quad 1 \leq j \leq n$$

最大子段和-动规

```
int MaxSubSum3(int n, int a[])
{
    
$$b[j] = \max\{b[j-1] + a_j, 0\} \quad 1 \leq j \leq n$$

    int sum=0, tms=0; //sum存储当前最大
    for(int j=1; j<=n; j++) {
        tms += a[j];
        if(tms<0) tms=0; //若和为负，则从下一个位置累和
        if(tms>sum) sum=tms; 例: (-1, 5, -3, 6, 4, 2, -7, 8)
    }
    return sum;
}
```

运行时间: $T(n)=O(n)$

引例

- ⑩ 某国为了防御敌国的导弹袭击，发展出一种导弹拦截系统。但是这种导弹拦截系统有一个缺陷：虽然它的第一发炮弹能够达到任意的高度，但是以后每一发炮弹都不能高于前一发的高度。某天，雷达捕捉到敌国的导弹来袭。由于该系统还在试用阶段，所以只有一套系统，因此有可能不能拦截所有的导弹。

引例

- ⑩ $D[i]$:表示第*i*枚导弹被拦截以后, 还能最多拦截的导弹数(包括被拦截的第*i*枚), 则

$$D[i] = \begin{cases} 1 & i = n \quad \text{or} \quad (i < j \leq n \text{ and } x(j) > x(i)) \\ \max\{D[j] + 1\} & i < n \quad (i < j \leq n \text{ and } x(j) \leq x(i)) \end{cases}$$

■ 例如, 60, 40, 45, 70, 42

<i>i</i>	5	4	3	2	1
$D[i]$	1	2	2	1	3